



SECURITY AUDIT

Lista Dao

SlisXAUE

FINAL REPORT

June 2026

BAILSEC.IO



Disclaimer:

Security assessment projects are time-boxed and reliant on information provided by the client, its affiliates, or its partners. The findings in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase, and apply only to the specific commit hash, version, or deployment reviewed at the time of the audit. Any subsequent modification, redeployment, or upgrade is outside the scope of this report and may invalidate its findings.

The audit does not constitute investment, legal, financial, regulatory, or tax advice, nor an endorsement of any project or team. It is based on a limited review of materials provided and is delivered on an "as-is," "where-is," and "as-available" basis. Use of blockchain technology is subject to unknown risks and flaws. The scope of this assessment is technical and contract-level; unless explicitly stated otherwise, it does not cover economic or game-theoretic risks, MEV, oracle manipulation, governance attacks, centralization or admin-key risks, off-chain components, front-end code, or the behavior of third-party protocols, dependencies, libraries, or services interacted with by the audited code.

We make no warranties, express or implied, regarding the accuracy, reliability, completeness, or availability of the report or any associated services, and disclaim all implied warranties including merchantability, fitness for a particular purpose, and non-infringement. We assume no responsibility for any product, service, software, code, or material referenced by, linked to, or accessible through the report.

This report is prepared solely for the contract owner and may not be relied upon by any third party. The contract owner is responsible for making their own decisions based on the report and should seek additional professional advice as needed. To the maximum extent permitted by law, our total aggregate liability arising from or related to this audit shall not exceed the fees actually paid for the engagement. The contract owner agrees to indemnify and hold harmless the audit firm and its personnel from any claims, damages, expenses, or liabilities arising from use of or reliance on the report or the audited smart contract.

This disclaimer and any dispute arising from the audit shall be governed by the laws of Wyoming (USA), without regard to its conflict-of-laws principles. By engaging in this audit, the contract owner acknowledges and agrees to these terms.

1. Project Details

Important: Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao – SlisXAUE – Audit Report
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/lista-new-contracts/pull/27/changes/3dd605e6322fa797e515dd8c5f85cb370c6c22b8
Resolution 1	https://github.com/lista-dao/lista-new-contracts/pull/27/changes/8c407a1f2976bf2bf1ad6612764e0e1f8a36b301

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged [no change made]	Failed resolution	Open
High	3	3				
Medium	7	3	1	3		
Low	14	4	1	9		
Informational	14	1		13		
Governance	2			2		
Total	40	11	2	27		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

Global

Issue_01	Governance Privilege: System reliance
Severity	Governance
Description	Currently, the contract exposes one or more governance privileges which can result in unexpected operations and potential loss of funds. The architecture fully relies on governance correctness, including governance within the CoboFundToken.
Recommendations	Consider implementing a strong multisig setup for all privileged addresses in the system. Consider monitoring governance actions within the CoboFundToken.
Comments / Resolution	Acknowledged.

slisXAUE

The SlisXAUE contract is an upgradeable ERC20 token which represents shares within the XAUTStaking contract and is minted during deposit and burned during requestWithdraw.

Privileged Functions

- mint
- burn

Invariants

INV 1: Only the XAUTStaking contract can mint and burn tokens

Issue_02	Governance Privilege: Minting and burning
Severity	Governance
Description	Currently, the contract exposes one or more governance privileges which can result in unexpected operations and potential loss of funds. Governance can mint and burn tokens.
Recommendations	Consider implementing a strong multisig setup for all privileged addresses in the system.
Comments / Resolution	Acknowledged.

XAUEAdapter

The `XAUEAdapter` contract is used as an adapter contract between the `XAUTStaking` contract and the `CoboFundToken` (`XAUEFundToken`). It essentially receives the XAUT asset from the `XAUTStaking` contract and then deposits it into the `CoboFundToken` contract to receive a share receipt. It is expected that the underlying oracle price increases which then makes the share receipt token more valuable in terms of XAUT. Any

Appendix: Profit mechanism

Whenever the oracle price increases, this will increase the NAV and as such, profit can be realized via the `updateVaultAssets` function which then determines the profit via the difference of `getVaultTotalAssets` and `lastVaultTotalAssets`. A `feeRate` is then applied on any profit and is assigned to the fee state variable which is later claimable by governance. Any post-fee amount is routed to the `XAUTStaking` contract where it increases the `totalAssets` value (unless there are no stakers)

Appendix: Loss mechanism

Whenever the oracle price decreases, this will decrease the NAV and as such, loss can be realized via the `updateVaultAssets` function which then determines the loss via the difference of `getVaultTotalAssets` and `lastVaultTotalAssets`. Unlike with profit distribution, no `feeRate` is applied and the loss is routed completely to the `XAUTStaking` contract.

Appendix: NAV mechanism

The NAV (net-asset-value) describes the value of the underlying shares and is simply the multiplication of the shares with the oracle value:

```
> expectedShareBalance × nav / 1e30
```

In a nutshell, it simply returns the XAUT equivalent for the currently owned shares and is influenced by the oracle return value.



Privileged Functions

- depositToVault
- requestWithdrawFromVault
- finishEarnPoolWithdraw
- claimFee
- setFeeReceiver
- setFeeRate
- setMaxDeltaBps
- setStaking
- emergencyWithdraw
- acknowledgeReject

Issue_03	Fee can result in insolvency
Severity	High
Description	The issue occurs in the loss path of XAUEAdapter._pushLoss. When <code>slisXAUE.totalSupply() == 0</code>, <code>_pushLoss</code> reduces fee by as much of the loss as possible. However, when <code>slisXAUE.totalSupply() > 0</code>, <code>_pushLoss</code> does not use fee as a loss buffer at all. It forwards the entire loss to <code>XAUTStaking.decreaseTotalAssets</code>. <code>XAUTStaking.decreaseTotalAssets</code> then caps the reduction at the current <code>userTotalAssetsScaled</code> to avoid underflow. If the NAV loss is larger than the remaining active user accounting, only the user accounting is reduced to zero. The unabsorbed part of the loss is not returned to the adapter and is not deducted from fee. After that, <code>_updateVaultAssets</code> still writes <code>lastVaultTotalAssets = newVaultTotalAssets</code>. The terminal state is therefore inconsistent: * <code>lastVaultTotalAssets</code> reflects the lower post-loss vault value; * <code>userTotalAssetsScaled</code> may be zero; * <code>slisXAUE.totalSupply()</code> may still be greater than zero; * fee remains unchanged even though the vault backing that economically supported it has been reduced. The root cause is that loss allocation depends only on whether <code>slisXAUE.totalSupply()</code> is zero, not on whether active user accounting is sufficient to absorb the loss. Once user accounting is exhausted, the remaining loss should not simply disappear while fee remains fully claimable. Initial state after prior gains: * <code>lastVaultTotalAssets = 150</code> * <code>XAUTStaking.totalAssets() = 100</code>

	* XAUEAdapter.fee = 50 * slisXAUE.totalSupply() > 0 A user withdraws 96 XAUT, and BOT requests redemption for 96 XAUT from XAUE. After the redemption request: * lastVaultTotalAssets = 54 * XAUTStaking.totalAssets() = 4 * XAUEAdapter.fee = 50 * slisXAUE.totalSupply() > 0 Now the XAUE vault position suffers a 10% loss. * New vault value: 48.6 * Loss: 5.4 * This is within the default maxDeltaBps = 1000 cap because $5.4 / 54 = 10\%$. XAUEAdapter._pushLoss(5.4, 54) sees nonzero slisXAUE.totalSupply() and sends the full 5.4 loss to XAUTStaking.decreaseTotalAssets. XAUTStaking.decreaseTotalAssets can only reduce 4 XAUT of active user accounting, so it caps the reduction: * XAUTStaking.totalAssets() = 0 * XAUEAdapter.fee = 50 * lastVaultTotalAssets = 48.6 The adapter now records a 50 XAUT fee claim while the remaining tracked vault value is only 48.6 XAUT. The excess 1.4 XAUT fee claim is not backed by the remaining vault position.
Recommendations	The current code allows for profit and losses via NAV sync but the canonical CoboFundToken oracle currently only supports increases which means the loss path is never triggered. If we follow the

	assumption and remove the loss path (ie. revert vault updates on loss), this issue could be considered as fully fixed. Oracle reference: https://etherscan.io/address/0xde7d4d8cac1a8cb6a70b3232517fd0ec3f950cdf#code
Comments / Resolution	Resolved, the loss path has been removed and it is expected that the oracle remains monotonically increasing.

Issue_04	Phantom holder value can create insolvency
Severity	High
Description	The adapter reconciles NAV movement onto the staking side: gains are split with fee and the remainder pushed via <code>increaseTotalAssets</code>, while losses are pushed in full via <code>decreaseTotalAssets</code>. The staking conversion denominator <code>userTotalAssetsScaled</code> is the holder-side ledger, and the share price (<code>convertToAssets</code>) reads it directly. Crucially, <code>getVaultTotalAssets/lastVaultTotalAssets</code> value the entire adapter position, which simultaneously backs the holders, the accrued fee, and, for the window between a <code>requestWithdraw</code> (which decrements <code>userTotalAssetsScaled</code> and burns shares) and the BOT's <code>requestWithdrawFromVault</code> (which redeems the backing XAUE shares), a queued withdrawal whose backing is still in the vault but no longer counted in <code>userTotalAssetsScaled</code>. This split between "what the vault is worth" and "what the holder ledger tracks" is the load-bearing surface. <code>decreaseTotalAssets</code> caps the scaled reduction at the current <code>userTotalAssetsScaled</code>, so a loss larger than the holder ledger floors it to 0 and discards the surplus, whereas <code>increaseTotalAssets</code> is uncapped. The incorrect assumption is that the whole vault loss belongs to holders, in fact the loss attributable to fee and to any in-flight queued-withdrawal backing is also dumped onto the holder ledger. When that non-holder value is large relative to the stake, even a modest, in-cap decline exceeds the stake, wipes it, and drops the excess; the invariant <code>lastVaultTotalAssets = stake + fee (+ in-flight queue backing)</code> breaks by the dropped amount. A recovery then credits holders the full non-fee gain from the zero base, value their ~0 vault share does not entitle them to, so a net-zero round-trip leaves total claims above the vault. It is reachable through ordinary NAV volatility against a fee-heavy (or mid-withdrawal) vault, and existing logic does not prevent it because

	the floor never re-applies the dropped loss and nothing reconciles the holder ledger plus fee against the actual vault value. Impact 1: Phantom holder value / staking insolvency: after a decline-then-recovery, <code>userTotalAssetsScaled</code> (and fee) sum to more than the vault holds; the share price over-values holders' shares by the dropped-loss amount. Impact 2: First-out advantage: holders who withdraw first realize the phantom value at full price, leaving the last withdrawers and the fee under-funded against the diminished vault. Impact 3: Boundary arming: the wipe (<code>userTotalAssetsScaled = 0</code> with <code>totalSupply > 0</code>) is exactly the precondition for the M-08 deposit/withdraw brick, so the accounting corruption can also freeze the protocol.
Recommendations	The current code allows for profit and losses via NAV sync but the canonical CoboFundToken oracle currently only supports increases which means the loss path is never triggered. If we follow the assumption and remove the loss path (ie. revert vault updates on loss), this issue could be considered as fully fixed. Oracle reference: https://etherscan.io/address/0xde7d4d8cac1a8cb6a70b3232517fd0ec3f950cdf#code
Comments / Resolution	Resolved, the loss path has been removed and it is expected that the oracle remains monotonically increasing.

Issue_05	Delta check update brick during full fee claim in edge-case
Severity	Medium
Description	<code>XAUEAdapter</code> tracks XAUE vault exposure through <code>expectedShareBalance</code> and values it through <code>getVaultTotalAssets()</code>. The returned value is denominated in XAUT raw units and is calculated by multiplying expected XAUE shares by NAV, then dividing by $1e30$. When the protocol has accrued fees but no active stakers, gains are stored in <code>XAUEAdapter.fee</code>. To actually pay those fees, the adapter must hold raw XAUT. A natural operational path is for BOT to redeem XAUE shares through <code>requestWithdrawFromVault</code>, wait for XAUE approval, and then call <code>claimFee</code> to transfer the raw XAUT to <code>feeReceiver</code>. The full-redemption path is sensitive to rounding because <code>getVaultTotalAssets()</code> rounds the vault value down to XAUT raw units, while <code>requestWithdrawFromVault()</code> computes the share amount needed to redeem that rounded XAUT amount. This can redeem enough shares to receive the rounded asset amount while still leaving a tiny positive XAUE share residual inside <code>expectedShareBalance</code>. The issue occurs when BOT tries to realize the full fee-backed vault value. <code>XAUEAdapter.getVaultTotalAssets()</code> computes the current value of <code>expectedShareBalance</code> and rounds down to XAUT raw units. If BOT then calls <code>requestWithdrawFromVault(assetAmount)</code> with <code>assetAmount</code> equal to that full rounded value, the adapter computes the XAUE shares to redeem using ceiling rounding. That ceiling is only sufficient to redeem the requested rounded XAUT amount. It is not guaranteed to burn the entire <code>expectedShareBalance</code>. If the original vault value contained a fractional XAUT raw unit, a residual share amount can remain. The residual is positive in share units, but its current XAUT value rounds down to zero.

After the redemption request, `requestWithdrawFromVault` writes:

- `expectedShareBalance = residualShares`
- `lastVaultTotalAssets = getVaultTotalAssets()`

Because the residual shares are worth less than one XAUT raw unit, `lastVaultTotalAssets` becomes 0.

The broken state is:

- `expectedShareBalance > 0`
- `lastVaultTotalAssets == 0`

At this point, `claimFee` may transfer the redeemed raw XAUT and reduce fee, but the residual share state remains.

If NAV later increases even slightly, those residual shares can become worth 1 XAUT raw unit. A later `_updateVaultAssets()` then computes:

```
* newVaultTotalAssets = 1
* lastVaultTotalAssets = 0
* gain = 1
* baseValue = 0
```

`_pushInterest` checks the delta cap before the zero-supply fee branch. Since the base value is zero, any positive gain fails the cap check. This makes the update revert.

The root cause is that the adapter allows a positive `expectedShareBalance` to be checkpointed with `lastVaultTotalAssets == 0`, but the delta-cap logic cannot process a later transition from zero value to positive value.

Initial state:

- `slisXAUE.totalSupply() = 0`

	• <code>XAUEAdapter.fee = 999,999</code> XAUT raw units • <code>XAUEAdapter.expectedShareBalance = 952,380,952,380,952,380</code> • <code>XAUEAdapter.getVaultTotalAssets() = 999,999</code> • <code>XAUEAdapter.lastVaultTotalAssets = 999,999</code> BOT requests redemption for the full rounded vault value: • <code>requestWithdrawFromVault(999,999)</code> The adapter computes the required shares with ceiling rounding: • shares redeemed: <code>952,380,000,000,000,000</code> • residual expected shares: <code>952,380,952,380</code> The residual shares are still positive, but their current value rounds down to zero: • <code>getVaultTotalAssets() = 0</code> • <code>lastVaultTotalAssets = 0</code> • <code>expectedShareBalance = 952,380,952,380</code> XAUE later approves the redemption, and BOT calls <code>claimFee(999,999)</code>. The fee is paid out, but the residual expected shares remain. Now NAV increases slightly. The residual shares round up to 1 XAUT raw unit: • <code>newVaultTotalAssets = 1</code> • <code>lastVaultTotalAssets = 0</code> • <code>gain = 1</code> The next <code>updateVaultAssets()</code> reverts because the delta check evaluates a positive gain against a zero base.
Recommendations	Since this is an unlikely edge-case and a code-fix might introduce new issues, we recommend ensuring that governance executes the first deposit and always ensures that a state with <code>lastVaultTotalAssets = 0</code> and <code>expectedShareBalance > 0</code> cannot happen. Furthermore, we refer to the general recommendation about potential negative exchange rates.

Comments / Resolution

Partially resolved, the delta check is now omitted if the base value is zero. It has to be highlighted that small rounded base values with corresponding large profit can still revert, e.g.:

- `baseValue = 1, gain = 1` → 100% gain, fails.
- `baseValue = 2, gain = 1` → 50% gain, fails.
- `baseValue = 3, gain = 1` → 33.33% gain, fails.

This can happen for example when there is a state with:

no stakers, BOT has redeemed XAUE shares to fund a full fee claim, leaving a sub-1-XAUT expectedShareBalance and lastVaultTotalAssets = 0.

NAV accrues; a routine permissionless updateVaultAssets[] books the 0 → 1 transition through the C2 skip and sets lastVaultTotalAssets = 1.

NAV accrues further until getVaultTotalAssets[] reaches 2, the next sync computes gain = 1 against baseValue = 1 and reverts "delta exceeds max." MANAGER raises maxDeltaBps to the 3000 ceiling; the sync still reverts (10000 > 3000). From this point any new depositor's deposit call (which syncs first) also reverts, so the protocol cannot be revived by ordinary use and only a UUPS upgrade restores it.

We still recommend removing the maximum delta in case of such edge-cases so the protocol can continue function without upgrade.

Furthermore we still recommend governance to execute (and retain) the first deposit.

Issue_06	Lack of vault update during <code>claimFee</code> allows fee claiming to bypass downside for fee decrease in <code>totalSupply = 0</code> state
Severity	Medium
Description	<code>claimFee</code> pays accrued protocol fee in XAUT from the adapter's own balance, gated only by <code>fee >= feeAmount</code> and <code>balanceOf(asset) >= feeAmount</code>, it does not call <code>_updateVaultAssets</code>. The fee variable is grown by NAV gains in <code>_pushInterest</code> and, uniquely when there are no <code>slisXAUE</code> holders, shrunk by NAV losses in the <code>_pushLoss</code> no-holder branch (<code>fee -= min[loss, fee]</code>), where fee serves as the junior buffer that absorbs losses before they reach the adapter principal backing already-queued withdrawals. Because both the buffer role and the claim run off the same fee accumulator, the order of "recognize the loss" versus "claim the fee" determines who bears a pending drawdown. Since <code>claimFee</code> does not force a sync, the fee it reads is stale in exactly one reachable state: <code>totalSupply == 0</code> with a NAV loss that has occurred on the oracle but not yet been pushed through <code>_updateVaultAssets</code>. A sync-first ordering would route that loss into the no-holder <code>_pushLoss</code> branch and reduce fee accordingly; claiming first instead extracts fee at its full pre-loss value, after which the later sync finds little or no fee left to absorb the loss, so the residual falls onto the adapter's XAUT principal that backs the queued (already-burned) withdrawal requests. The incorrect implicit assumption is that the fee is current at claim time. In every other state the omission is harmless: un-synced gains have not yet been added to fee (the claim is conservative), and at <code>totalSupply > 0</code> losses never touch fee at all. Nothing prevents the harmful ordering because the only guard is <code>fee >= feeAmount</code> against the unsynced accumulator.
Recommendations	The current code allows for profit and losses via NAV sync but the canonical CoboFundToken oracle currently only supports increases which means the loss path is never triggered. If we follow the assumption and remove the loss path (ie. revert vault updates on loss), this issue could be considered as fully fixed.

	Oracle reference: https://etherscan.io/address/0xde7d4d8cac1a8cb6a70b3232517fd0ec3f950cdf#code
Comments / Resolution	Resolved, the loss path has been removed and it is expected that the oracle remains monotonically increasing.

Issue_07	MEV-window around <code>acknowledgeReject</code> allows for flash-thefting part of potential profits
Severity	Medium
Description	When XAUE rejects a pending redemption, the BOT calls <code>acknowledgeReject</code>, which re-credits the returned shares as principal at the request-time XAUT value (<code>lastVaultTotalAssets += lockedAssetAmount</code>, <code>expectedShareBalance += shareAmount</code>) and performs no NAV sync. The resulting in-flight delta, the difference between the slice's current value and its request-time value, surfaces on the next <code>_updateVaultAssets</code>, where <code>_pushInterest</code> sends a gain to current slisXAUE holders if any exist, or assigns it entirely to fee when <code>totalSupply == 0</code>. The realizing sync is permissionless (any deposit, withdrawal, or <code>updateVaultAssets</code> call), and the ack itself is preceded by XAUE's public <code>RedemptionRejected</code> event, so the timing of both the arming and the realization is observable. The deferred gain is distributed at a later, permissionless moment and its destination (holders versus fee) is decided by the <code>totalSupply</code> gate at that moment, not at the ack. An actor who becomes a holder before the ack is therefore a holder when the post-ack sync distributes the gain, and captures a pro-rata share. The deposit-side "sync-first" front-run mitigation does not prevent this: at the time the attacker deposits, the rejected slice is still excluded from <code>expectedShareBalance</code>, so the leading <code>updateVaultAssets()</code> realizes nothing; the ack arms the gain afterward, and the attacker's shares already exist when it is paid out. Furthermore, at <code>totalSupply == 0</code>, the gain would otherwise go to fee, so a one-wei deposit before the ack makes the attacker the sole holder and reroutes essentially the whole deferred gain to them. Existing logic does not block it because nothing ties the gain's realization to the holder set that existed when the rejected slice was originally in the vault.

Recommendations	There is no simple solution for this issue. We recommend only allowing governance to call this function manually with a private RPC to ensure attackers cannot detect it.
Comments / Resolution	Acknowledged.

Issue_08	Immediate NAV impact from <code>acknowledgeReject</code> can result in update DoS
Severity	Medium
Description	<code>acknowledgeReject</code> handles XAUE rejecting a previously requested redemption. It re-credits the returned shares as principal at the request-time XAUT value, doing <code>expectedShareBalance += shareAmount</code> and <code>lastVaultTotalAssets += lockedAssetAmount</code>, where <code>lockedAssetAmount</code> is the <code>assetAmount</code> XAUE stored at <code>requestRedemption</code> time. It intentionally performs no <code>_updateVaultAssets</code>; the developers expect any resulting NAV delta to surface on the next sync and pass through the standard fee/cap path, and they expect that delta to be small because the BOT heartbeat keeps per-sync moves small and the XAUE oracle moves at most a few percent per day. The per-update guard <code>delta * 10000 <= baseValue.maxDeltaBps</code> (<code>maxDeltaBps</code> default 1000, ceiling 3000) is the load-bearing safety the whole sync relies on, and it is built on the assumption that every delta it sees is a single incremental move. The assumption breaks specifically for the rejected slice. From <code>requestWithdrawFromVault</code> until the rejection is acknowledged, that slice's shares are absent from <code>expectedShareBalance</code>, so <code>getVaultTotalAssets</code> never reflects them and their NAV drift over the full pending window is never synced. <code>acknowledgeReject</code> then bumps <code>lastVaultTotalAssets</code> by the stale request-time value while the shares are now worth the current value, so the next sync computes a delta equal to the entire accumulated drift on that slice in one shot. When the slice is large relative to the remaining position and NAV has drifted by more than the cap over the pending period, that single delta exceeds <code>baseValue.maxDeltaBps</code> – and can exceed even the 3000-bps ceiling – so <code>_pushLoss</code> (or <code>_pushInterest</code>, if NAV rose) reverts, taking <code>_updateVaultAssets</code> and every caller with it.
Recommendations	Consider increasing or removing the maximum settable value for deviations and then manually intervene via a deviation increase.

Comments / Resolution	Acknowledged.
-----------------------	---------------

Issue_09	Funds below <code>minRedeemShares</code> cannot be withdrawn and may deadlock the last batch
Severity	Medium
Description	It can regularly happen that the shares held by the <code>XAUEAdapter</code> get below <code>minRedeemShares</code>. In this situation, shares cannot be redeemed because it reverts within the <code>CoboFundToken</code>. This issue has been escalated to medium severity because it can legitimately block a full batch. If for example a batch requires 1000 tokens to redeem and there are 990 tokens in the adapter and 10 tokens in the <code>CoboFundToken</code>, which might be below the <code>minRedeemShare</code> value, these tokens can never be withdrawn and as such the batch is deadlocked.
Recommendations	Consider making sure that the BOT never withdraws an amount which results shares to fall below the <code>minRedeemShares</code> .
Comments / Resolution	Acknowledged.

Issue_10	Drop of <code>totalVaultAssets</code> to zero with subsequent NAV increase will DoS updates.
Severity	Medium
Description	The <code>XAUEAdapter</code> reconciles NAV movement on every sync, valuing its XAUE holdings as <code>getVaultTotalAssets() = expectedShareBalance · nav / 1e30</code> (6-dec XAUT). <code>_updateVaultAssets</code> compares this against <code>lastVaultTotalAssets</code> and routes any difference through <code>_pushInterest/_pushLoss</code>, each gated by the anomaly cap <code>delta · 10000 ≤ baseValue · maxDeltaBps</code> with <code>baseValue = lastVaultTotalAssets</code>. Because every user deposit and <code>requestWithdraw</code> invokes this sync first, the value of <code>lastVaultTotalAssets</code> is load-bearing for protocol liveness. <code>expectedShareBalance</code> shrinks whenever the BOT redeems XAUE shares for an exiting staker via <code>requestWithdrawFromVault</code>, so a near-full exit at low TVL drives it toward dust, and <code>getVaultTotalAssets</code> is an integer floor, so once <code>expectedShareBalance · nav < 1e30</code> it returns exactly 0. When <code>expectedShareBalance</code> lands just below the truncation edge, the sync that follows the exit records <code>lastVaultTotalAssets = 0</code>. The cap then has a zero base. On the next NAV increase that pushes <code>expectedShareBalance · nav</code> back to or above <code>1e30</code>, <code>getVaultTotalAssets</code> flips from 0 to a positive <code>k</code>, and the cap check becomes <code>k · 10000 ≤ 0</code>, which is false for any <code>k > 0</code>, so the push reverts and the whole <code>_updateVaultAssets</code> reverts. Every entry point that syncs first is now blocked. The defect is that the cap is expressed relative to a base that the truncation can drive to exactly zero, turning the anomaly guard into an unconditional revert.
Recommendations	The current code allows for profit and losses via NAV sync but the canonical CoboFundToken oracle currently only supports increases which means the loss path is never triggered. If we follow the assumption and remove the loss path (ie. revert vault updates on loss), this issue could be considered as fully fixed.

	Oracle reference: https://etherscan.io/address/0xde7d4d8cac1a8cb6a70b3232517fd0ec3f950cdf#code
Comments / Resolution	Resolved, the loss path has been removed and it is expected that the oracle remains monotonically increasing.

Issue_11	Vault update will brick during <code>forceRedeem</code>
Severity	Low
Description	The <code>forceRedeem</code> function on the <code>CoboFundToken</code> burns shares from any recipient and can also do so from the <code>XAUEAdapter</code> . If such a situation ever happens, shares are being burned but <code>expectedShareBalance</code> remains unsynced and as such the vault update will break.
Recommendations	We recommend a sync function which allows governance to reconcile this state. It should only ever called if this unlikely situation ever occurs
Comments / Resolution	Acknowledged.

Issue_12	MAX_DELTA_BPS_CAP can result in permanent DoS
Severity	Low
Description	The MAX_DELTA_BPS_CAP value is hardcoded at 30% and as such prevents from setting a higher limit. It is theoretically possible to exceed this (e.g. if the contract is paused). In that scenario, it would permanently DoS the contract.
Recommendations	Consider removing this limitation and ensure governance/BOT is trusted.
Comments / Resolution	Acknowledged.

Issue_13	Unclaimed fee principal creates yield for users
Severity	Low
Description	XAUEAdapter tracks the protocol's XAUE share exposure through expectedShareBalance and values it through getVaultTotalAssets(). When the vault value increases, _pushInterest compares the current vault value against lastVaultTotalAssets and allocates the gain. If there are no slisXAUE holders, _pushInterest treats the entire gain as protocol-owned and increases XAUEAdapter.fee. This is consistent with the idea that, while no users are staked, any gain on remaining adapter-held XAUE shares belongs to the protocol. However, the accumulated fee is only an accounting claim. Unless BOT redeems and claims it, the corresponding value can remain inside the adapter's XAUE share position. That means protocol fee principal remains included in expectedShareBalance, lastVaultTotalAssets, and future NAV gain calculations. The issue is that XAUEAdapter does not separate the XAUE shares or NAV value backing accrued protocol fees from the XAUE shares or NAV value backing active stakers. The accounting branch in _pushInterest is based only on slisXAUE.totalSupply(): * if total supply is zero, the full gain goes to fee; * if total supply is nonzero, only feeRate of the gain goes to fee, and the rest goes to stakers. That means, if there is only 1 wei of stakes, the fee will produce yield for stakers.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_14	Maximum amount withdrawal may revert due to rounding
Severity	Low
Description	During <code>requestWithdraw</code>, the round-up is correct for protocol solvency but is applied unconditionally, with no clamp to the caller's actual balance. When amount is even one wei above the exact floor value of the full balance B (<code>amount > convertToAssets(B)</code>) yet still inside the band where <code>convertToShares(amount)</code> floors to exactly B, the round-up condition is satisfied and pushes shares to B + 1. The subsequent <code>require(balanceOf >= shares)</code> then reverts. The incorrect implicit assumption is that the rounded-up share count never exceeds what the caller can hold for their own max exit; for any amount above the floor that maps back to B, it does. It is reachable because a max-exit is naturally computed as "the asset value of my balance," and a <code>pricePerShare</code>-based estimate, an added safety margin, or a rate that the entry NAV sync moves between estimate and execution can all produce an amount in the reverting band
Recommendations	We recommend applying the proper rounding scenarios.
Comments / Resolution	Acknowledged.

Issue_15	Change of feeRate will have hindsight impact
Severity	Low
Description	Currently, whenever the feeRate is changed, this will not update the asset state and forward profit/loss. As such, a feeRate update will have hindsight effects.
Recommendations	Consider making it optional to enforce a vault update during the feeRate updated.
Comments / Resolution	Partially resolved, it is now enforced. A better solution would be to make it optional with a boolean parameter.

Issue_16	Immutable oracle might desync from CoboFundToken oracle
Severity	Low
Description	The oracle is currently set once and then immutable but within the CoboFundToken , the oracle is actually mutable. If now the oracle is changed, this will have multiple-side effects but most importantly it means that the XAUEAdapter will now use an outdated oracle which is not synced anymore.
Recommendations	Consider implementing a function which allows for the change of the oracle. It has to be noted that if such a situation ever occurs, it will very likely have general side-effects.
Comments / Resolution	Resolved. However, it enforces a vault update. A better solution would be to add a boolean which optionally updates the vault in case of edge-cases, one such edge-case would be that in case of a NAV decrease, the oracle cannot be changed anymore because it routes through vault asset update. Furthermore, a price increase between the old and new oracle will not be reflected as yield. A similar edge-case would apply if the price decreases. It can also expose side-effects due to acknowledgeReject logic. However, all of this is very unlikely as it is never expected to change the oracle. If such a change is desired, we recommend the allocation of additional security resources before execution.

Issue_17	Fee receiver change can redirect accrued fees
Severity	Low
Description	In <code>XAUEAdapter</code>, accrued fees are stored in <code>fee</code>, and <code>claimFee</code> sends them to the current <code>feeReceiver</code>. The issue is that <code>setFeeReceiver</code> changes <code>feeReceiver</code> immediately, but it does not force a fee claim first. Because <code>claimFee</code> (BOT) and <code>setFeeReceiver</code> (MANAGER) are called by different actors, they cannot be guaranteed to run atomically. If both transactions are sent around the same time, mempool ordering decides whether already-accrued fees go to the old receiver or the new one. This can cause fee flow to go to a different receiver than intended.
Recommendations	Consider adding the option to claim pending fees before changing the receiver in <code>setFeeReceiver</code> .
Comments / Resolution	Acknowledged.

Issue_18	Unsynced max-delta change can freeze sync-first flows
Severity	Low
Description	<code>setMaxDeltaBps</code> changes the delta cap immediately, but it does not sync vault assets first. Because of this, the new cap is applied to the already accumulated unsynced delta between <code>'lastVaultTotalAssets'</code> and the current vault value. If the manager lowers <code>'maxDeltaBps'</code> below that backlog move, the next <code>'_updateVaultAssets()'</code> call reverts with <code>'delta exceeds max'</code>. This can block flows that call sync first, including <code>'deposit()'</code> and <code>'requestWithdraw()'</code> in <code>'XAUTStaking'</code>, since both call <code>'adapter.updateVaultAssets()'</code> before continuing. The result is a manager cap reduction can temporarily freeze core user flows until the cap is raised again.
Recommendations	Consider adding the option to call <code>_updateVaultAssets</code> before setting the new value in <code>setMaxDeltaBps</code> .
Comments / Resolution	Resolved. However, a better option would be to add a boolean.

Issue_19	Attacker can deliberately delay batch confirmation
Severity	Informational
Description	An attacker can delay a batch confirmation if it is about to be confirmed but a full day has not yet passed, executing a redemption to increase the required amount for that batch which then can result in that batch not being redeemable yet.
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged.

Issue_20	Side-effects of emergencyWithdraw
Severity	Informational
Description	The emergencyWithdraw function will withdraw funds without accounting change. This will have second-order-effects such as silent insolvency.
Recommendations	Consider only using this function if absolutely required.
Comments / Resolution	Acknowledged.

Issue_21	Share balance <-> expectedShareBalance deviation during rejected-not-yet-acknowledged state
Severity	Informational
Description	Whenever the CoboFundToken rejects a withdrawal request it will transfer the shares back to the initiator. If the acknowledgeReject function is not triggered, it will now have a larger share balance than the expectedShareBalance value. This is not an issue but can potentially become a precondition.
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged.

Issue_22	<code>CoboFundToken.forceRedeem</code> will result in invariant break
Severity	Informational
Description	One important invariant is that the owned share balance is larger than <code>expectedShareBalance</code> and this is enforced on every update. If the <code>forceRedeem</code> function on the <code>CoboFundToken</code> contract is triggered, it will burn shares without decreasing <code>expectedShareBalance</code>, resulting in a DoS for future vault updates.
Recommendations	Consider keeping this in mind.
Comments / Resolution	Acknowledged.

Issue_23	Oracle updated can be frontrun to extract yield
Severity	Informational
Description	Oracle updates most of the time rely on external triggers which can be detected in the mempool. In such a situation if a large price increase is expected, a user can simply execute a deposit and capture a part of that profit
Recommendations	Consider keeping that in mind.
Comments / Resolution	Acknowledged.

Issue_24	Frequent update calls will truncate fee
Severity	Informational
Description	Each update call will calculate the protocol fee which incurs truncation. Frequent calls will thus truncate more often than infrequent calls and will result in smaller protocol fee.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_25	<code>depositToVault</code> does not pre-check XAUE <code>minDepositAmount</code>
Severity	Informational
Description	<code>XAUEAdapter.depositToVault(assetAmount)</code> forwards <code>assetAmount</code> straight to <code>IXAUEFundToken.mint(assetAmount)</code> without first checking it against the external <code>minDepositAmount()</code> floor. A dust <code>assetAmount</code> therefore reverts deep inside the external XAUE call rather than at the adapter boundary. The behaviour is also inconsistent with <code>requestWithdrawFromVault</code> , which does pre-validate against the external <code>minRedeemShares()</code> floor (" <code>below xaue minRedeem</code> ").
Recommendations	Read <code>IXAUEFundToken.minDepositAmount()</code> and <code>require(assetAmount >= that, "below xaue minDeposit")</code> at the top of <code>depositToVault</code> , mirroring the existing min-redeem guard in <code>requestWithdrawFromVault</code> .
Comments / Resolution	Resolved.

Issue_26	No event distinguishes fee-absorbed loss from silently-dropped loss
Severity	Informational
Description	When <code>_pushLoss</code> runs with <code>totalSupply == 0</code>, the adapter's <code>fee</code> accumulator absorbs the loss up to its balance and any residual is silently dropped against adapter principal. In all cases <code>_updateVaultAssets</code> emits only <code>VaultLoss(newVaultTotalAssets, totalLoss)</code>; the gain path by contrast emits <code>UpdateVaultAssets</code> with both fee and net components. There is no on-chain signal that separates the fee-absorbed portion from the silently-dropped residual, leaving off-chain accounting unable to reconcile where a no-holder loss went.
Recommendations	Emit a dedicated event from <code>_pushLoss</code> (e.g. <code>LossAbsorbed(feeAbsorbed, residualDropped)</code>) in the <code>totalSupply == 0</code> branch so the split is observable on-chain.
Comments / Resolution	Acknowledged.

Issue_27	MIN_WITHDRAW sits below XAUE's live redeem floor, stranding min-size burned requests
Severity	Informational
Description	<code>requestWithdrawFromVault</code> computes <code>shareAmount = ceil(assetAmount * 1e30 / nav)</code> and requires <code>shareAmount >= IXAUEFundToken.minRedeemShares()</code> [1e18]. At the live NAV the smallest redeemable XAUT amount is <code>[1e18 * nav / 1e30] ≈ 1003</code> wei and grows as NAV accrues, which exceeds the configured <code>XAUTStaking.MIN_WITHDRAW</code> [1000 wei]. A user can deposit <code>MIN_DEPOSIT</code> and pass both staking-side gates (<code>>= minDeposit</code> on deposit, <code>>= minWithdraw</code> on <code>requestWithdraw</code> – <code>slisXAUE</code> is burned), yet the BOT can never issue the redemption to fund the batch: <code>requestWithdrawFromVault</code> reverts "below xaue minRedeem".
Recommendations	Set MIN_WITHDRAW to dominate the live redeem floor with margin (<code>>= ceil(minRedeemShares * navMax / 1e30)</code>), or derive it dynamically from <code>minRedeemShares</code> and current NAV at request time so a passing staking-side request is always redeemable downstream.
Comments / Resolution	Acknowledged.

XAUTStaking

The XAUTStaking contract is a simple asynchronous vault which allows users to deposit XAUT tokens and receive share tokens as slisXAUE. The underlying XAUT tokens are then immediately transferred to the XAUEAdapter where they can be deposited into the CoboFundToken contract to accrue yield via oracle price increases. Any oracle price increase or decrease is translated into the XAUTStaking contract via adjusting the `userTotalAssetsScaled` storage variable.

This follows the standard ERC4626 mechanism where shares are corresponding to assets and it is usually expected that the underlying share value increases over time which results in yield for stakers.

The withdraw mechanism is asynchronous because funds are not sitting in the XAUTStaking contract but instead either in the XAUEAdapter or within the CoboFundToken contract. It is therefore require to signal an irreversible withdrawal intent via `requestWithdraw`

Appendix: Share Value Change

Whenever the XAUEAdapter updates the vault assets, this will either call the `increaseTotalAssets` or `decreaseTotalAssets` function which then in turn changes the underlying share value.

Appendix: Asynchronous Withdrawals

Assets are immediately forwarded to the XAUEAdapter contract and as such the XAUTStaking contract never holds any assets and incorporates the XAUEAdapter into the primary trust assumptions to seed withdrawals.

Users can request withdrawals via the `requestWithdraw` function which immediately burns shares and decreases the `userTotalAssetsScaled` value. It is then expected that the XAUEAdapter invokes the `finishWithdraw` function which potentially confirms batches, if the transferred withdrawal amount + `withdrawQuote` can cover the requested amount. The `withdrawQuota` storage variable is a moving variable which buffers funds that are transferred into the contract but cannot satisfy the amount for the latest batch.

Whenever the corresponding batch is then confirmed, users can claim their withdraw requests via the `claimWithdraw` function.

Privileged Functions

- setMintCap
- setMinDeposit
- setMinWithdraw
- emergencyWithdraw
- pause
- unpause

Invariants:

INV 1: Before deposits and withdrawals, total assets must be updated

INV 2: Deposits must never result in zero amounts or shares

INV 3: Deposits must not exceed the mintCap

INV 4: Withdrawals must increment batch if one day has passed or the currentBatchId is confirmed

INV 5: Withdrawals can only be claimed if the corresponding batch has been confirmed

INV 6: withdrawQuota must always assign the rolling amount beyond confirmed batches

INV 7: Withdrawal requests must immediately burn shares and decrease userTotalAssetsScaled

Issue_28	NAV decrease after requested withdrawal will result in insolvency
Severity	High
Description	<code>XAUTStaking</code> lets users exit by calling <code>requestWithdraw</code>. This burns the user's <code>slisXAUE</code> immediately, records a fixed XAUT-denominated withdrawal request, increases <code>totalWithdrawAmountInBatch[currentBatchId]</code>, and decreases <code>userTotalAssetsScaled</code>. The actual XAUT needed to pay those requests is not redeemed from XAUE during <code>requestWithdraw</code>. Instead, BOT later calls <code>XAUEAdapter.requestWithdrawFromVault</code>, waits for XAUE redemption approval, then calls <code>XAUEAdapter.finishEarnPoolWithdraw</code>, which forwards XAUT to <code>XAUTStaking.finishWithdraw</code>. <code>finishWithdraw</code> does not partially confirm withdrawal batches. It confirms batches only while <code>withdrawQuota</code> is at least the full <code>totalWithdrawAmountInBatch[i]</code>. If the quota is smaller than the next batch amount, the loop stops and the batch remains unconfirmed. This means fixed queued withdrawal liabilities must remain aligned with the amount that can actually be redeemed from the adapter's XAUE position. The issue occurs because withdrawal requests are fixed at the staking exchange rate when <code>requestWithdraw</code> is called, but the underlying XAUE shares backing those requests can remain exposed to NAV movement until BOT performs the actual XAUE redemption. The failure sequence is: 1. A user requests withdrawal. 2. Their shares are burned. 3. A fixed XAUT amount is stored in <code>userWithdrawalRequests</code>. 4. The batch total is increased in <code>totalWithdrawAmountInBatch</code>. 5. <code>userTotalAssetsScaled</code> is decreased.

	6. The adapter's XAUE exposure is not reduced yet. 7. NAV later increases. 8. The gain is pushed to the remaining active stakers through <code>increaseTotalAssets</code>. 9. A remaining holder withdraws at the increased exchange rate, creating an additional fixed claim. 10. NAV later decreases before the adapter redeems enough XAUT. 11. The loss cannot reduce already queued withdrawal requests. 12. If all shares have been burned, <code>slisXAUE.totalSupply() == 0</code>, so the staking-side accounting no longer has active holders to absorb the loss. 13. BOT can only redeem the lower current vault value. 14. <code>finishWithdraw</code> receives less XAUT than the fixed queued batch total. 15. The batch is not confirmed, so <code>claimWithdraw</code> remains blocked. The root cause is that the protocol treats <code>requestWithdraw</code> as a final XAUT liability creation event, but the economic asset backing that liability is only redeemed later and can lose value before redemption settlement.
Recommendations	The current code allows for profit and losses via NAV sync but the canonical CoboFundToken oracle currently only supports increases which means the loss path is never triggered. If we follow the assumption and remove the loss path (ie. revert vault updates on loss), this issue could be considered as fully fixed. Oracle reference: https://etherscan.io/address/0xde7d4d8cac1a8cb6a70b3232517fd0ec3f950cdf#code
Comments / Resolution	Resolved, the loss path has been removed and it is expected that the oracle remains monotonically increasing.

Issue_29	share inflation will DoS the contract
Severity	Medium
Description	XAUTStaking.deposit prices a deposit by shares = <code>convertToShares[amount]</code> and then enforces <code>require{totalSupply + shares <= mintCap}</code> before minting. <code>convertToShares</code> uses virtual offsets: $[amount \cdot SCALE_RATIO] \cdot [totalSupply + 1] / [userTotalAssetsScaled + 1]$ so that the first deposit and the zero-asset state remain well-defined. <code>mintCap</code> is a MANAGER risk control intended to bound total share issuance. The load-bearing state variable is <code>userTotalAssetsScaled</code>: it is the conversion denominator, and the loss path [<code>decreaseTotalAssets</code>, floored at the available balance] can drive it to 0 while <code>totalSupply</code> remains large, because losses are socialized entirely onto stakers and never onto the accrued fee. Once <code>userTotalAssetsScaled</code> is floored to 0 with a large <code>totalSupply</code>, the conversion denominator becomes the virtual +1, so <code>convertToShares[amount]</code> scales with <code>[totalSupply + 1]</code> and explodes for any non-trivial deposit. The subsequent <code>require{totalSupply + shares <= mintCap}</code> then fails, reverting the deposit. The incorrect implicit assumption is that the share-price formula stays well-conditioned across all reachable states; at the <code>userTotalAssetsScaled → 0, totalSupply ≫ 0</code> boundary it does not, and the <code>mintCap</code> guard converts the instability into an unconditional revert. It is reachable because that boundary is produced by ordinary bounded NAV losses against a fee-heavy / donation-heavy vault, and existing logic does not prevent it, the only state that would re-condition the formula [<code>userTotalAssetsScaled</code> regrowing via new deposits] is exactly what is blocked.
Recommendations	The current code allows for profit and losses via NAV sync but the

	canonical CoboFundToken oracle currently only supports increases which means the loss path is never triggered. If we follow the assumption and remove the loss path (ie. revert vault updates on loss), this issue could be considered as fully fixed. Oracle reference: https://etherscan.io/address/0xde7d4d8cac1a8cb6a70b3232517fd0ec3f950cdf#code
Comments / Resolution	Resolved, the loss path has been removed and it is expected that the oracle remains monotonically increasing.

Issue_30	<code>minWithdraw</code> check check is based on assets which can decrease
Severity	Low
Description	The <code>minWithdraw</code> check is based on assets and even if users have deposited above the <code>minWithdraw</code> amount, an exchange rate decrease can push the amount below the <code>minWithdraw</code> amount which would then DoS a legitimate withdrawal and requires a user to top-up their deposit. Likewise, a partial withdrawal can result in such a situation.
Recommendations	To mitigate this issue as isolated as possible while keeping the check in place we recommend making the <code>minDeposit</code> value significantly higher than the <code>minWithdraw</code> value and additionally add documentation to the <code>requestWithdraw</code> function which highlights the risk of partial withdrawals.
Comments / Resolution	Acknowledged.

Issue_31	Partial withdrawals can result in amount below <code>minWithdraw</code>
Severity	Low
Description	As the title already describes, it can happen that a user accidentally withdraws below the <code>minWithdraw</code> amount which means that the residual amount cannot be withdrawn until a new deposit is executed.
Recommendations	Consider keeping this in mind.
Comments / Resolution	Acknowledged.

Issue_32	1 wei is sufficient to prevent full fee capturing
Severity	Low
Description	The XAUEAdapter contract short circuits any profit/loss distribution if there are no shares for the XAUTStaking contract. If the XAUTStaking contract is empty, a clever user can simply deposit a specific amount, then withdraw a specific amount and leave 1 wei of dust. This dust would then accumulate profit/loss.
Recommendations	Consider executing a governance deposit at the beginning and never withdraw these funds. This will fully mitigate this issue.
Comments / Resolution	Acknowledged.

Issue_33	Async withdrawal will result in yield for remaining holders
Severity	Low
Description	Whenever the requestWithdraw function is triggered, it will immediately burn shares and decrease userTotalAssetsScaled but it will not adjust the actual lastVaultTotalAssets value within the XAUEAdapter. As such, any pending funds will keep on contributing to yield accrual but will not capture it and the yield or loss is translated to remaining holders. This mechanism can serve as a precondition for more advanced, unexpected states.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged.

Issue_34	Rounding is incorrect
Severity	Low
Description	The <code>convertToAssets</code> and <code>convertToShares</code> function both round down and as safeguard, the required asset amount is rounded up within <code>requestWithdraw</code>. This behavior is incorrect and the logic should follow the standard ERC4626-pattern which rounds into the correct direction in the first place.
Recommendations	Consider adjusting the rounding behavior.
Comments / Resolution	Resolved. Notable, the shares-input branch computes <code>amount = convertToAssets(shares)</code> and, when <code>convertToShares(amount) < shares</code>, sets <code>amount += 1</code> before <code>IERC20(asset).safeTransferFrom(msg.sender, adapter, amount)</code>. A caller who pre-computed cost off-chain as <code>convertToAssets(shares)</code> and approved/holds exactly that amount will hit a revert whenever the bump fires

Issue_35	<code>pricePerShare</code> returns a stale price between BOT heartbeats
Severity	Low
Description	<code>pricePerShare()</code> calls <code>convertToAssets(1e18)</code> directly, reading <code>userTotalAssetsScaled</code> as currently stored without first syncing the adapter NAV. All state-changing paths (<code>deposit</code> , <code>requestWithdraw</code>) call <code>IXAUEAdapter(adapter).updateVaultAssets()</code> before pricing, so the live execution price reflects accrued NAV – but the view does not. Off-chain integrators reading <code>pricePerShare</code> between BOT heartbeats therefore receive a stale price that lags the true value by the unbooked NAV growth, which can mislead displays, routers, or collateral valuations.
Recommendations	Document that <code>pricePerShare</code> is a last-synced snapshot, or expose a companion view that computes the price against <code>getVaultTotalAssets()</code> (which reads live oracle NAV) so consumers can obtain the up-to-date value without a state change.
Comments / Resolution	Resolved.

Issue_36	Side-effects of <code>emergencyWithdraw</code>
Severity	Informational
Description	The <code>emergencyWithdraw</code> function will withdraw funds without accounting change. This will have second-order-effects such as silent insolvency.
Recommendations	Consider only using this function if absolutely required.
Comments / Resolution	Acknowledged.

Issue_37	Value accrual from virtual share
Severity	Informational
Description	The contract exposes 1 wei of virtual shares and these shares will accrue value which means after all users have withdrawn, totalSupply = 0 and userTotalAssetsScaled > 0. This can become a precondition for advanced issues.
Recommendations	Consider ensuring that governance always does the first deposit and never withdraws to prevent the explained state.
Comments / Resolution	Acknowledged.

Issue_38	Dead code within <code>increaseTotalAssets/decreaseTotalAssets</code>
Severity	Informational
Description	Both aforementioned functions expose a condition which is triggered when totalSupply == 0. However, since this function is only callable via the adapter and the adapter already short circuits in that scenario: <pre>if (IERC20[slisXAUE].totalSupply() == 0) { fee += gain; return [gain, 0]; }</pre> This condition is basically unreachable.
Recommendations	Consider keeping it in mind. We don't recommend removing it because it might be used in the future if a different adapter is used.
Comments / Resolution	Acknowledged.

Issue_39	<code>requestWithdraw</code> checks only the requested amount, leaving a residual position below <code>minWithdraw</code>
Severity	Informational
Description	<code>requestWithdraw</code> enforces <code>require(amount >= minWithdraw, "below min withdraw")</code> on the requested amount but never checks the user's remaining balance after the burn. A partial withdrawal can leave a residual position worth less than <code>minWithdraw</code> , which can then never be withdrawn on its own (any future request for it fails the same <code>minWithdraw</code> gate). The dust is permanently locked unless topped back up above the threshold.
Recommendations	After computing the burn, require that the user's remaining share value is either zero or <code>>= minWithdraw</code> (reject requests that would strand a sub-minimum residual), or allow a full-balance withdrawal path that bypasses the minimum.
Comments / Resolution	Acknowledged.

Issue_40	<code>finishWithdraw</code> is one-way; surplus XAUT stranded in <code>withdrawQuota</code> cannot be returned to the adapter
Severity	Informational
Description	<code>XAUTStaking.finishWithdraw</code> only moves XAUT into staking; there is no reverse path back to the adapter. XAUT can exit staking only via <code>claimWithdraw</code> (to users) or <code>emergencyWithdraw</code> (MANAGER). Any surplus from bulk pre-funding, request-time rounding, or fee buffers, or any XAUT stranded by NAV-window drift between a redemption request and its approval, accumulates in <code>withdrawQuota</code> with no clean way to recycle it back into the adapter to be re-deployed for yield.
Recommendations	Add a MANAGER/BOT function to transfer surplus XAUT (the balance above outstanding confirmed-but-unclaimed obligations) from staking back to the adapter, so idle settlement surplus can be re-deployed instead of sitting dormant.
Comments / Resolution	Acknowledged.